

Milk & Cookies

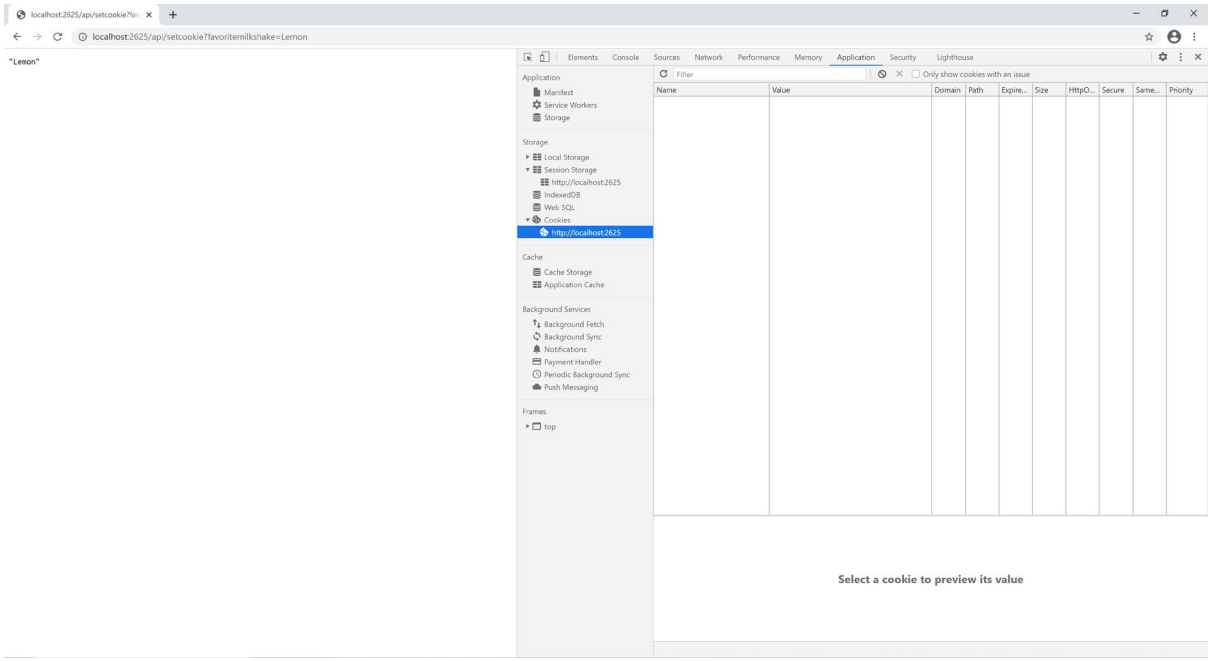
Sessioner og cookies C#

I denne opgave skal du arbejde med begreberne cookies og sessions. Opgaven er opdelt i mange små opgaver, som har til formål at du får afprøvet nogle af de interessante muligheder der ligger i brugen af sessioner og cookies.

Opret en nyt web-Api projekt og brug det igennem din opgavebesvarelse. Det er hurtigere og nemmere end at oprettet et website og principperne er de samme. Det anbefales at du opretter en controller til hver opgave, for overskuelighedens skyld, og primært benytter GET metoden, hvis intet andet angives.

Du kan via GET metoden sende parametre med på requestet, som en querystring. Når du gør det, skal din GET metode tage argumenter med.

Brug Chrome til at undersøge dine cookies – det gøres ved at åbne udviklerværktøjet, vælg applikation -> storage og efterfølgende Cookies



Opgave 1:

a) Opret en forespørgsel, hvor du sender navnet på din ynglings Milkshake med som parameter Eks: **http://localhost:2625/api/[navn på din controller]?favoritemilkshake=Lemon**

   Lad controlleren returnere, din yndlingsdrik som en streng.

b) Udvid nu din controller ved at sætte en cookie. I din cookie, sætter du yndlingsdrikken. Du sætter en ny cookie, ved at bruge append metoden.

```
Response.Cookies.Append("favoriteMilkshake", favoriteMilkshake);
```

   Undersøg i Chrome, at din cookie er blevet sat

c) Nu skal du hente data fra cookien i stedet for din querystring. Implementer en ny GET metode, men denne gang uden parametre. Hent værdien i favoriteMilkshake fra din cookie og returner denne – husk at tjekke om din cookie er null!!!! Hvis der ikke er nogen værdi i din Cookie, skal du bare returnerer unknown.

Man henter data ved at bruge index notationen

```
Request.Cookies["favoriteMilkshake"]
```

Man kan ikke bare have to metoder som begge hedder GET, du er derfor nødt til at ændre navnet på én af dine metoder og sætte en "action" som annotation.

Du kan implementere den nye GET metode, men giv den et andet navn end GET. Angiv dit metodenavn som action i attributten ovenover din metodesignatur.

```
[HttpGet]
[Route("[action]")]
public string GetFromCookie()
{
    …
}
```

d) Du lagde måske mærke til at din cookie forsvandt, hver eneste gang du stoppede web-serveren i opgave C. Det skete fordi at dit browservindue blev lukket ned og at cookien kun er sat til at leve, så længe sessionen er i gang.

Juster nu din kode, til at sætte expiry til 5 minutter fra nu!

For at kunne gøre dette, skal du oprette et objekt hvor du kan sætte en række options på en cookie og sætte parametrene med når du appender din cookie.

```
CookieOptions co = new CookieOptions();

   … implementer +5 minutter her

Response.Cookies.Append("favoriteMilkshake", favoriteMilkshake, co);
```

e) Opret to nye controllers med hver deres route. Tilføj noget data på den ene cookie, som du kan returnere. Sæt en path som hører til denne controller. I den anden controller – som har en anden route, skal du prøve at hive data fra cookien, kan du det?

f) Som en sidste ting, skal du lære at rydde op efter dig. Den eneste måde man kan slette en Cookie på manuelt, er ved at sætte datoen fra før end (now). Prøv at fjerne din cookie med enten expires eller max-age

Skelet til opgave 1

```
[Produces("application/json")]
[Route("api/SetCookie")]
 public class SetCookieController : Controller
 {
    // GET: api/SetCookie
    [HttpGet]
     public string Get(string favoriteMilkshake)
     {
         //do stuff her
         return favoriteMilkshake;
     }

     // GET: api/SetCookie
     [HttpGet]
     [Route("[action]")]
     public string GetFromCookie()
     {
       return //Something here;
     }
 }
```

Opgave 2:

Sessioner virker stort set på samme måde, som ved Cookies – bortset fra at data her gemmes på serversiden.

I denne opgave skal du bruge to browsere og du skal implementere en ”lille” indkøbsvogn og alt styres via session / cookies.

Microsoft har i deres Core framework ændret en del på den måde de håndtere Sessioner på, fremfor tidligere versioner.

Så inden du går i gang med din implementering, skal du have installeret et par nuget pakker – hvis du ikke allerede har dem installeret.

Følg denne vejledning : https://benjii.me/2016/07/using-sessions-and-httpcontext-in-aspnetcore-and-mvc-core/

Når du skal gemme objekter i din session, kræver det lidt mere arbejde, end bare de simple datatyper. Det kræver at data serialiseres til et bytearray.

Du kan f.eks. oprette en klasse med nedenstående kode. Der er en extension metode som du kan bruge på din Session.

```
public static class SessionExtensions
{
    public static void SetObjectAsJson(this ISession session, string key,
object value)
    {
        session.SetString(key, JsonConvert.SerializeObject(value));
    }

    public static T GetObjectFromJson<T>(this ISession session, string key)
    {
        var value = session.GetString(key);
        return value == null ? default(T) :
JsonConvert.DeserializeObject<T>(value);
    }
}
```

Du kan herefter tilføje objekter til din session

```
var myComplexObject = new MyClass();
HttpContext.Session.SetObjectAsJson("Test", myComplexObject);
```

a) Opret en klasse, produkt som repræsenterer et Name og en Price. Vi skal bruge denne klasse til at lægge vare ned i "indkøbskurven"

b) Opret en ny controller (Shoppingcart) og implementer en GET metode, som tager 2 parametre, et productName og en Price. GET metoden skal returnere en liste (`IEnumerable`) af Product.

c) Sessionen virker som en "vault", hvor du kan lægge objekter ned i baseret på en specifik klient. Så det allerførste du skal gøre når der ankommer et request, er at tjekke om indkøbskurven allerede er oprettet og ligger nede på sessionen. Hvis kurven eksisterer, skal det produkt som sendes med via querystring lægges ned i kurven. Hvis ikke indkøbskurven eksisterer oprettes denne og produktet tilføjes og efterfølgende lægges kurven ned på sessionen.

d) Prøv lige at kigge ned i din cookie-beholder…. Nu skulle du gerne have fået en ny session cookie!!

e) Det er ikke så nemt at simulere en DELETE funktion gennem et browserkald med http delete. Men hvis man gerne vil fjerne et element fra kurven, kunne det være en oplagt mulighed at implementere denne metode. Opret i stedet en ny controller, som kan slette en vare fra kurven, hvis den eksisterer. Få opdateret din session.

f) Afprøv nu dit system i en anden browser – du vil se at data tilhører hver klient.

g) Luk din browser ned og afprøv systemet igen ..hvad skete der med din session?